Recorder

Updated: 2018-09-06

An abstract base component that saves snapshots of any kind at regular intervals to enable rewinding.

The recorder class is just a shell to properly record snapshots and rewind by interpolating between them. By itself, it doesn't do anything; you can't add it to a GameObject. However, it can be extended (subclassed) to record any type of information in snapshots.

Methods

These methods are also available on the following timeline components when `Timeline.rewindable` is enabled:

- `Timeline.transform` (for objects without a rigidbody)
- `Timeline.rigidbody` (for objects with a 3D rigidbody)
- `Timeline.rigidbody2D` (for objects with a 2D rigidbody)

```
void ModifySnapshots(SnapshotModifier modifier)
```

Modifies all snapshots via the specified modifier delegate.

```
void Record()
```

Forces the recorder to record a snapshot at the current frame.

Examples

Move all recorded snapshots up by 1 unit when the up arrow is pressed:

```
using UnityEngine;
using Chronos;

class MyScript : MonoBehaviour
{
    public void Update()
    {
        if (Input.GetKeyDown(KeyCode.UpArrow))
        {
            Timeline timeline = GetComponent<Timeline>();

            timeline.transform.ModifySnapshots((snapshot, snapshotTime) =>
            {
                snapshot.position += new Vector3(0, 1, 0);

                return snapshot;
            });
        }
    }
}
```

Custom Recorders

Creating a custom recorder from the editor

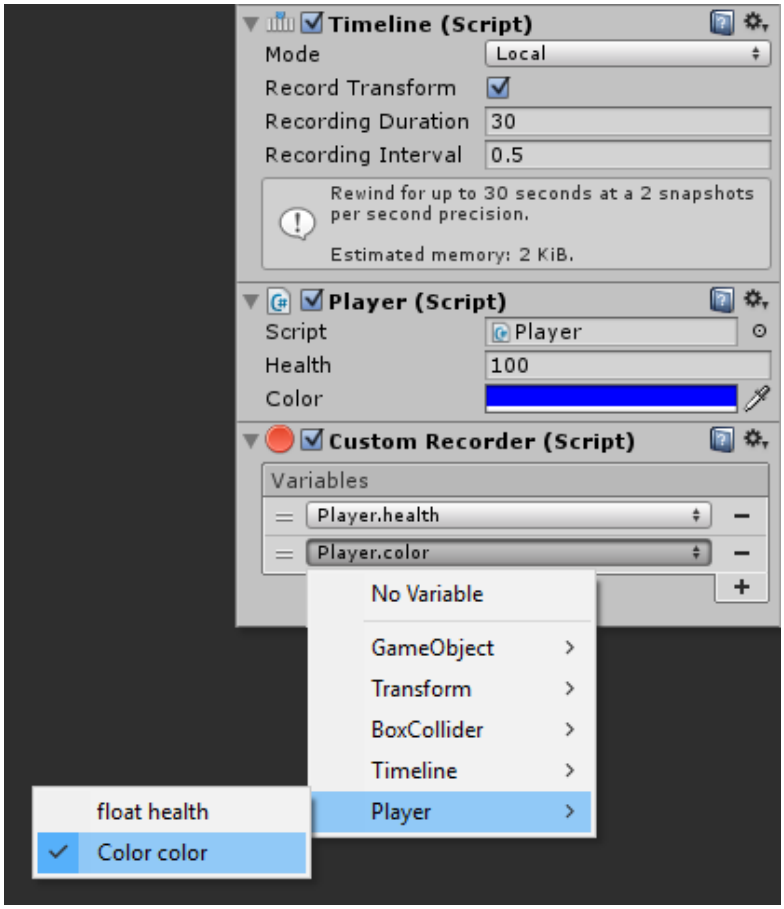
Let's say we want to record the health and color of our player (presuming its color changes over time). Our player script would look similar to this:

```
using UnityEngine;

public class Player : MonoBehaviour
{
    public float health = 100;
    public Color color = Color.blue;

    void Update()
    {
        // ...
    }
}
```

All we have to do is add a `Custom Recorder` component to our game object, and specify which variables to record:



That's all there is to it! Rewinding will now properly handle the player's health and color.

Custom recorders support all fields and properties that are value-typed and not read-only. They work on built-in Unity components and even custom scripts.

Creating a custom recorder from script

Creating a recorder from scripting is a tiny bit more complex, but it's more efficient in terms of speed and memory. If you are targetting mobiles, you might want to consider making your recorders from scripting before releasing, while still using editor-based custom recorders for fast prototyping.

Again, let's record the health and color of our player. We will create a `PlayerRecorder` script that inherits `Recorder`, and implement the 3 mandatory abstract methods:

- `CopySnapshot`: Records the current state of the object and returns a snapshot
- `ApplySnapshot`: Takes a snapshot and applies it to the object
- `LerpSnapshot`: Interpolates between two snapshots and returns the result

```
using UnityEngine;
using Chronos;

// Make your class inherit Recorder.
// The generic parameter points to the type of snapshot.
```

```
public class PlayerRecorder : Recorder<PlayerRecorder.Snapshot>
{
    // The struct that contains each of our snapshot's data.
    // In our case, a health value and a color.
    public struct Snapshot
    {
        public float health;
        public Color color;
    }

    // Record the current health and color in a snapshot
    protected override Snapshot CopySnapshot()
    {
        return new Snapshot()
        {
            health = GetComponent<Player>().health,
            color = GetComponent<Player>().color,
        };
    }

    // Apply a snapshot to the current GameObject
    protected override void ApplySnapshot(Snapshot snapshot)
    {
        GetComponent<Player>().health = snapshot.health;
        GetComponent<Player>().color = snapshot.color;
    }

    // Interpolate between two snapshots for smoothing
    protected override Snapshot LerpSnapshots(Snapshot from, Snapshot to, float t)
    {
        return new Snapshot()
        {
            health = Mathf.Lerp(from.health, to.health, t),
            color = Color.Lerp(from.color, to.color, t)
        };
    }
}
```

You can then attach your `PlayerRecorder` component, along with a timeline, to your player GameObject. Its health and color should now be rewindable.

Previous

← **Occurence**

An event anchored at a specified moment in time composed of two actions: one when time goes forward, and another when time goes backward.

Was this article helpful?

Be the first to vote!

Yes, helpful | No, not for me

ludiq

PRODUCTS

LUDIQ

Contact

Blog

Copyright © 2022 Ludiq. All Rights Reserved. • • Terms of Service • Icons by IconMonstr and FatCow